

## COMPTE-RENDU DE PROJET

# Plateforme Intelligente de Risque Crédit, Détection de Fraude & Analyse de Marchés

*Data Science, MLOps & Déploiement Cloud*

Réaliser par : Youssef Eljaouhary

07 juillet 2026

# Table des matières

|                                                                                |   |
|--------------------------------------------------------------------------------|---|
| Table des matières .....                                                       | 2 |
| 1. Contexte et objectifs du projet .....                                       | 3 |
| 1.1 Objectifs fonctionnels .....                                               | 3 |
| 1.2 Objectifs pédagogiques et professionnels .....                             | 3 |
| 2. Architecture globale de la plateforme .....                                 | 3 |
| 2.1 Flux de traitement .....                                                   | 3 |
| 2.2 Organisation du dépôt .....                                                | 3 |
| 3. Jeux de données .....                                                       | 4 |
| 4. Modules Machine Learning & Deep Learning réalisés .....                     | 4 |
| 4.1 Credit Scoring .....                                                       | 4 |
| 4.2 Détection de fraude .....                                                  | 4 |
| 4.3 Segmentation clientèle (Clustering) .....                                  | 4 |
| 4.4 Analyse de sentiment financier (NLP) .....                                 | 5 |
| 4.5 Prédiction boursière (LSTM) .....                                          | 5 |
| 5. Feature Engineering & Analyse Exploratoire (EDA) .....                      | 5 |
| 5.1 Feature Engineering (src/feature_engineering.py) .....                     | 5 |
| 5.2 Analyse exploratoire (notebooks/01_EDA_et_Feature_Engineering.ipynb) ..... | 5 |
| 6. Persistance des données (Base SQLite) .....                                 | 5 |
| 7. Interface utilisateur — Centre de Décision .....                            | 6 |
| 7.1 Principes de design .....                                                  | 6 |
| 7.2 Modules du dashboard .....                                                 | 6 |
| 8. API FastAPI & Frontend découplé .....                                       | 6 |
| 9. Tests et qualité logicielle .....                                           | 7 |
| 10. Stratégies de déploiement .....                                            | 7 |
| 11. Stack technique récapitulative .....                                       | 8 |
| 12. Compétences démontrées .....                                               | 8 |
| 13. Conclusion et perspectives .....                                           | 9 |
| Pistes d'évolution .....                                                       | 9 |

## 1. Contexte et objectifs du projet

Le secteur bancaire et financier fait face à des enjeux croissants de gestion du risque : évaluer la solvabilité des emprunteurs, détecter les fraudes en temps réel, comprendre les comportements clients et anticiper les mouvements de marché. Ce projet répond à ces enjeux par une plateforme d'intelligence artificielle unifiée, couvrant l'ensemble du cycle de vie d'un projet de Data Science — de la préparation des données jusqu'au déploiement en production.

### 1.1 Objectifs fonctionnels

- Prédire si un client remboursera son prêt (Credit Scoring).
- Détecter les transactions frauduleuses en temps réel.
- Segmenter les clients selon leurs comportements financiers (Customer Segmentation).
- Analyser automatiquement les actualités et commentaires financiers (NLP).
- Prévoir les cours boursiers (Time Series Forecasting).
- Déployer la solution sur des infrastructures cloud accessibles à tous.

### 1.2 Objectifs pédagogiques et professionnels

Au-delà de la réalisation technique, le projet démontre une maîtrise de bout en bout : préparation des données, modélisation, évaluation, industrialisation (API, base de données), interface utilisateur orientée décision métier, conteneurisation, intégration continue et déploiement multi-cloud.

---

## 2. Architecture globale de la plateforme

L'architecture suit un principe de séparation claire des responsabilités : une couche de données et de modèles réutilisables (src/), exposée à la fois par une interface décisionnelle (Streamlit) et par une API REST (FastAPI), le tout persisté dans une base de données et orchestré via des outils MLOps standards.

|              |                                            |                                                   |
|--------------|--------------------------------------------|---------------------------------------------------|
| Données      | Génération / ingestion des jeux de données | Pandas, NumPy, générateur synthétique             |
| Modélisation | Entraînement et évaluation des modèles     | Scikit-learn, XGBoost, LightGBM, PyTorch          |
| API          | Exposition des modèles en service web      | FastAPI, Pydantic, Uvicorn                        |
| Frontend Web | Application web découplée consommant l'API | Next.js / React, déploiement Vercel               |
| Déploiement  | Hébergement accessible publiquement        | Hugging Face Spaces, Oracle Cloud, Render, Vercel |

### 2.1 Flux de traitement

Données brutes → Préparation & Feature Engineering → Analyse exploratoire (EDA) → Entraînement des modèles (Credit Scoring, Fraude, Clustering, NLP, LSTM) → Persistance (SQLite) → Exposition (API FastAPI) → Interface (Dashboard Streamlit / Frontend Next.js) → Conteneurisation (Docker) → Déploiement (Cloud).

### 2.2 Organisation du dépôt

- src/ — modules Python réutilisables (données, feature engineering, modèles, base de données)

- api/ — API FastAPI exposant les modèles en service web
- dashboard/ — Centre de décision Streamlit (thème Azure Profond)
- notebooks/ — notebook d'analyse exploratoire exécuté
- models/ — artefacts de modèles entraînés (.joblib, .pt)
- data/ — données brutes, segmentation, base SQLite
- tests/ — suite de tests unitaires (pytest)
- Fichiers de déploiement — Dockerfile, docker-compose.yml, render.yaml, DEPLOYMENT\*.md

### 3. Jeux de données

Un générateur de données synthétiques (src/data\_generation.py) reproduit fidèlement la structure statistique des jeux de données Kaggle de référence, permettant l'exécution complète du projet sans dépendance à une authentification externe. Le remplacement par les données réelles ne nécessite aucune modification du code de modélisation, uniquement le chemin du fichier source.

|                     |                                     |                |                                   |
|---------------------|-------------------------------------|----------------|-----------------------------------|
| Credit Scoring      | Give Me Some Credit (Kaggle)        | ~20 000 lignes | SeriousDlqin2yrs (défaut à 2 ans) |
| Segmentation client | Bank / Mall Customer Segmentation   | ~5 000 clients | Non supervisé (4 profils)         |
| Séries temporelles  | Yahoo Finance / Kaggle Stock Market | ~1 500 jours   | Cours de clôture (Close)          |

## 4. Modules Machine Learning & Deep Learning réalisés

### 4.1 Credit Scoring

Quatre algorithmes sont entraînés et comparés automatiquement : Régression Logistique, Random Forest, XGBoost et LightGBM. Le meilleur modèle (ROC-AUC) est sélectionné et sauvegardé.

- Feature engineering dédié : ratios financiers, transformation logarithmique, binning de l'âge, indicateur de risque structurel.
- Évaluation : Accuracy, Precision, Recall, F1-score, ROC-AUC.
- Fonction de prédiction unitaire exposée à l'API et au dashboard.

### 4.2 Détection de fraude

Approche hybride combinant apprentissage supervisé et détection d'anomalies, pour couvrir aussi bien les patterns de fraude connus que les comportements inédits :

- XGBoost (supervisé) — pondéré pour gérer le déséquilibre extrême des classes (< 0,2 % de fraude).
- Isolation Forest (non supervisé) — détection d'anomalies sans étiquette.
- AutoEncoder (Deep Learning, PyTorch) — entraîné uniquement sur transactions légitimes ; une erreur de reconstruction élevée signale une fraude potentielle.

### 4.3 Segmentation clientèle (Clustering)

Quatre algorithmes de clustering sont comparés (K-Means, DBSCAN, Agglomerative Clustering, Gaussian Mixture Model), évalués par Silhouette Score, Davies-Bouldin Index et Calinski-Harabasz Score. Le meilleur

algorithme est retenu, et les clusters sont nommés automatiquement selon leurs caractéristiques moyennes (revenu, solde, dépense) : Clients Premium, Fidèles, Occasionnels, à Risque.

#### 4.4 Analyse de sentiment financier (NLP)

Deux approches sont implémentées : un modèle rapide TF-IDF + Régression Logistique pour la production (utilisé par le dashboard et l'API), et un pipeline de fine-tuning BERT (DistilBERT via HuggingFace Transformers) pour une précision supérieure sur des données réelles plus nuancées.

#### 4.5 Prévision boursière (LSTM)

Un réseau de neurones récurrent LSTM (PyTorch) est entraîné sur une fenêtre glissante de 30 jours pour prédire le cours de clôture à J+1, avec une fonction de prévision itérative multi-jours. Évaluation par MAE et RMSE sur un ensemble de test temporel.

---

## 5. Feature Engineering & Analyse Exploratoire (EDA)

### 5.1 Feature Engineering (src/feature\_engineering.py)

Module réutilisable, partagé entre l'entraînement, l'API et le dashboard — garantissant que les transformations appliquées en production sont strictement identiques à celles validées à l'entraînement.

- Traitement des valeurs manquantes et plafonnement des outliers (IQR).
- Transformation logarithmique des variables asymétriques (revenu, montant).
- Ratios financiers et variables d'interaction (endettement × revenu, utilisation × retards...).
- Binning métier (tranches d'âge) et indicateurs de risque structurel.
- Weight of Evidence (WOE) et Information Value (IV) — standard du credit scoring pour évaluer le pouvoir prédictif d'une variable avant modélisation.
- Features temporelles pour la fraude (heure de la journée, indicateur nocturne, magnitude du vecteur PCA).
- Score d'engagement composite et ratios type RFM pour la segmentation client.
- Utilitaires génériques : rapport de valeurs manquantes, filtre de corrélation, importance de variables (Random Forest + information mutuelle).

### 5.2 Analyse exploratoire (notebooks/01\_EDA\_et\_Feature\_Engineering.ipynb)

Notebook Jupyter entièrement exécuté (25 cellules de code, 19 graphiques générés, aucune erreur), couvrant les 5 jeux de données : distributions, déséquilibre de classes, corrélations, WOE/IV, patterns temporels de fraude, relations entre variables clients, fréquence de mots par sentiment, rendements et volatilité boursière.

---

## 6. Persistance des données (Base SQLite)

Le module src/database.py introduit une couche de persistance : chaque interaction utilisateur (évaluation de crédit, vérification de transaction, analyse de sentiment, prévision boursière) est enregistrée. Les

indicateurs affichés sur le tableau de bord reflètent ainsi l'usage réel de la plateforme, et non uniquement les données de démonstration initiales.

| Table               | Contenu                                                                                 |
|---------------------|-----------------------------------------------------------------------------------------|
| credit_applications | Historique des dossiers de crédit évalués (probabilité de défaut, statut d'instruction) |
| fraud_alerts        | Historique des transactions analysées et alertes critiques                              |
| customer_segments   | Clients segmentés et géolocalisés (ville, latitude, longitude)                          |
| sentiment_log       | Historique des analyses de sentiment financier                                          |
| stock_forecasts     | Historique des prévisions boursières générées                                           |

Une fonction d'initialisation (`initialize_platform_db`) crée les tables et injecte un historique réaliste de démonstration (dossiers, alertes, segments géolocalisés sur des villes marocaines) au premier lancement.

---

## 7. Interface utilisateur — Centre de Décision

Le dashboard a été conçu selon une logique de centre de décision plutôt que de moniteur technique : navigation unifiée, indicateurs métiers plutôt que statuts système, et une structure identique dans chaque module.

### 7.1 Principes de design

- Thème « Azure Profond » avec bascule clair / sombre, sans emoji — icônes Bootstrap et indicateurs géométriques CSS.
- Cartes avec micro-interactions au survol (élévation, changement de teinte).
- Statuts exprimés en langage métier (« Opérationnel », « Risque élevé — crédit déconseillé ») plutôt qu'en jargon technique.
- Structure identique par module : Sommaire (résultat global) → Exploration (graphiques) → Action (tableau + export CSV).

### 7.2 Modules du dashboard

- Accueil — KPIs consolidés, aperçu des modules, notifications des alertes critiques.
- Octroi de Crédit — évaluation de dossier, historique, export des dossiers en attente.
- Surveillance des Transactions — analyse de transaction, alertes critiques à vérifier.
- Segmentation Clientèle — répartition des segments et carte géographique interactive des clients.
- Veille des Marchés — analyse de sentiment d'actualités et historique.
- Prévision Financière — prévision boursière LSTM et historique des prévisions.

---

## 8. API FastAPI & Frontend découplé

Une API REST documentée (Swagger / OpenAPI, accessible sous /docs) expose l'ensemble des modèles et de l'historique, avec gestion CORS pour permettre l'appel depuis un frontend hébergé sur un domaine distinct.

| Endpoint                                                                  | Méthode | Fonction                                            |
|---------------------------------------------------------------------------|---------|-----------------------------------------------------|
| /health, /kpis, /notifications                                            | GET     | État des services et indicateurs consolidés         |
| /predict/credit-scoring                                                   | POST    | Probabilité de défaut d'un dossier de crédit        |
| /predict/fraud                                                            | POST    | Probabilité de fraude d'une transaction             |
| /predict/sentiment                                                        | POST    | Sentiment d'une actualité financière                |
| /predict/stock-forecast                                                   | POST    | Prévision du cours boursier (horizon paramétrable)  |
| /segments/summary, /segments/customers                                    | GET     | Répartition et géolocalisation des segments clients |
| /credit/applications, /fraud/alerts, /sentiment/history, /stock/forecasts | GET     | Historique consultable pour le frontend             |

Un frontend web découplé (Next.js / React) est en cours d'intégration, consommant cette API pour offrir une expérience utilisateur autonome, déployable indépendamment du dashboard Streamlit (architecture backend/frontend séparée, typique des applications web modernes).

## 9. Tests et qualité logicielle

- Suite de tests unitaires (pytest) couvrant la génération de données, le feature engineering, le clustering et la base de données — 13 tests, tous passants.
- Tests d'intégration du dashboard via le framework Streamlit AppTest : chaque page et chaque action (évaluation, vérification, analyse) validées sans exception, y compris la carte de segmentation en mode clair et sombre.
- Vérification end-to-end de l'API (démarrage réel du serveur, appels sur chaque endpoint).
- Pipeline CI/CD (GitHub Actions) : tests automatiques → entraînement de contrôle → build et publication des images Docker.

## 10. Stratégies de déploiement

Plusieurs chemins de déploiement gratuits ont été documentés et préparés (fichiers Dockerfile, docker-compose.yml, render.yaml, guides pas-à-pas), afin de rendre la plateforme accessible publiquement selon le niveau de contrôle souhaité.

| Plateforme          | Ce qui est déployé  | Points forts                                                   |
|---------------------|---------------------|----------------------------------------------------------------|
| Hugging Face Spaces | Dashboard Streamlit | 16 Go de RAM gratuits, large marge pour torch/xgboost/lightgbm |

| Plateforme                | Ce qui est déployé                                           | Points forts                                                                                              |
|---------------------------|--------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Streamlit Community Cloud | Dashboard Streamlit                                          | Intégration GitHub la plus simple : 1 Go de RAM (dépendances allégées)                                    |
| Oracle Cloud Always Free  | Stack complète (API + Dashboard + MLflow via Docker Compose) | Vraie VM gratuite à vie (2 OCPU / 12 Go), stockage réellement persistant, HTTPS via Nginx + Let's Encrypt |
| Render                    | API FastAPI (backend)                                        | Déploiement Docker natif, CORS configuré pour un frontend distinct                                        |
| Vercel                    | Frontend Next.js (en intégration)                            | Déploiement optimisé pour les applications React/Next.js                                                  |

Chaque stratégie est documentée dans un guide dédié : DEPLOYMENT.md (Hugging Face Spaces / Streamlit Cloud), DEPLOYMENT\_ORACLE.md (Oracle Cloud Always Free) et DEPLOYMENT\_RENDER\_VERCEL.md (architecture backend Render / frontend Vercel).

## 11. Stack technique récapitulative

| Domaine          | Outils                                                             |
|------------------|--------------------------------------------------------------------|
| Données          | Pandas, NumPy                                                      |
| Machine Learning | Scikit-learn, XGBoost, LightGBM                                    |
| Deep Learning    | PyTorch (AutoEncoder, LSTM), Transformers (BERT)                   |
| Clustering       | K-Means, DBSCAN, Agglomerative Clustering, GMM, PCA/t-SNE/UMAP     |
| NLP              | TF-IDF, BERT (HuggingFace)                                         |
| Visualisation    | Plotly, Matplotlib, Seaborn, Streamlit                             |
| Base de données  | SQLite                                                             |
| API              | FastAPI, Pydantic, Uvicorn                                         |
| Frontend web     | Next.js / React                                                    |
| MLOps            | MLflow, Docker, GitHub Actions                                     |
| Cloud            | Hugging Face Spaces, Streamlit Cloud, Oracle Cloud, Render, Vercel |

## 12. Compétences démontrées

- Data Science — préparation, nettoyage et analyse des données.
- Machine Learning — classification, prédiction et segmentation.
- Deep Learning — LSTM, BERT, AutoEncoder.
- Feature Engineering avancé — WOE/IV, ratios métiers, binning.
- Ingénierie logicielle — architecture modulaire, tests automatisés, base de données.
- Conception d'interface — UX/UI orientée décision métier.

- Développement web — API REST, frontend découplé.
  - MLOps et Cloud — conteneurisation, CI/CD, déploiement multi-plateforme gratuit.
- 

## 13. Conclusion et perspectives

Ce projet constitue une solution complète de Data Science appliquée au secteur bancaire et financier, couvrant l'ensemble du cycle de vie d'un projet d'intelligence artificielle : de la préparation des données à un déploiement public gratuit, en passant par la modélisation, l'industrialisation et une expérience utilisateur pensée pour la prise de décision.

### Pistes d'évolution

- Basculer la base de données vers une solution managée persistante (PostgreSQL) pour un usage en production durable.
- Finaliser et déployer le frontend Next.js pour une expérience web autonome, indépendante du dashboard Streamlit.
- Ajouter un monitoring de dérive des données (data drift) pour les modèles en production.
- Mettre en place une authentification et une gestion des rôles pour un usage multi-utilisateurs.